

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

по «Алгоритмам и структурам данных»

Базовые задачи

Выполнил:

Студент группы Р3206

Косогова М. А.

Преподаватели:

Косяков М.С.

Тараканов Д. С.

Санкт-Петербург

2026

Задача №А «Агроном-любитель»

```
#include <iostream>
```

```
int N;
```

```
int iters = 1;
```

```
int prev = -1;
```

```
int flower;
```

```
int length = 0;
```

```
int main() {
```

```
    std::cin >> N;
```

```
    int j = 1;
```

```
    int index = 1;
```

```
    for (int i = 1; i <= N; i++) {
```

```
        std::cin >> flower;
```

```
        if (flower == prev) {
```

```
            iters++;
```

```
            if (iters == 3 && length < i - j) {
```

```
                index = j;
```

```
                length = i - j;
```

```
            }
```

```
        } else {
```

```
            if (iters >= 3) {
```

```
                j = i - 2;
```

```
            }
```

```
            prev = flower;
```

```
            iters = 1;
```

```
        }
```

```
    }
```

```

if (iters < 3 && length < N - j + 1) {
    index = j;
    length = N - j + 1;
}

```

```

std::cout << index << " " << index + length - 1 << std::endl;

```

```

return 0;

```

```

}

```

Пояснение к примененному алгоритму:

- 1) Идём по цветам, считаем повторы подряд;
- 2) Как только видим 3+ одинаковых подряд - запоминаем участок, если он самый длинный;
- 3) В конце выводим начало и конец самого длинного такого участка.

Задача №В «Зоопарк Глеба»

```

#include <iostream>

```

```

#include <vector>

```

```

bool is_uppercase(char ch) {
    return 'A' <= ch && ch <= 'Z';
}

```

```

bool are_same(char ch1, char ch2) {
    if (is_uppercase(ch1)) {
        return ch1 + 32 == ch2;
    } else {
        return ch1 - 32 == ch2;
    }
}

```

```

int main() {

```

```
std::ios_base::sync_with_stdio(false);
```

```
std::cin.tie(nullptr);
```

```
std::string s;
```

```
std::cin >> s;
```

```
std::vector<std::pair<char, int>> stack;
```

```
std::vector<int> indices(s.length() / 2);
```

```
int a = 0;
```

```
int b = 0;
```

```
for (int i = 0; i < static_cast<int>(s.length()); i++) {  
    if (!stack.empty() && are_same(stack.back().first, s[i])) {  
        if (is_uppercase(s[i])) {  
            indices[a++] = stack.back().second;  
        } else {  
            indices[stack.back().second] = b++;  
        }  
        stack.pop_back();  
    } else {  
        if (is_uppercase(s[i])) {  
            stack.push_back(std::make_pair(s[i], a++));  
        } else {  
            stack.push_back(std::make_pair(s[i], b++));  
        }  
    }  
}
```

```
if (stack.empty()) {
```

```
    std::cout << "Possible\n";
```

```
    for (int i = 0; i < static_cast<int>(s.length()) / 2; i++) {
```

```

        std::cout << indices[i] + 1 << " ";
    }
} else {
    std::cout << "Impossible";
}

std::cout << "\n";
return 0;
}

```

Пояснение к примененному алгоритму:

- 1) Читаем строку;
- 2) Идём по символам: ищем пары «заглавная+строчная» одной буквы через стек;
- 3) Если пара нашлась отмечаем её номера, убираем из стека; иначе — кладём символ в стек;
- 4) В конце: если стек пуст выводим Possible и номера пар; иначе — Impossible.

Задача №С «Конфигурационный файл»

```

#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(nullptr);

    std::unordered_map<std::string, std::vector<std::string>> mp;
    std::vector<std::vector<std::string>> tmp{ {} };
    std::pair<std::string, std::string> p;

    std::string s;

    while (std::cin >> s) {

```

```

if (s == "{") {
    tmp.push_back({});
} else if (s == "}") {
    for (int i = 0; i < static_cast<int>(tmp.back().size()); i++) {
        mp.at(tmp.back()[i]).pop_back();
    }
    tmp.pop_back();
} else {
    int i = static_cast<int>(s.find('='));
    p = std::make_pair(s.substr(0, i), s.substr(i + 1));

    if ('0' <= p.second.back() && p.second.back() <= '9') {
        if (mp.find(p.first) == mp.end()) {
            mp.insert({p.first, {p.second}});
            tmp.back().push_back(p.first);
        } else {
            mp.at(p.first).push_back(p.second);
            tmp.back().push_back(p.first);
        }
    } else {
        if (mp.find(p.second) == mp.end()) {
            mp.insert({p.second, {"0"}});
            tmp.back().push_back(p.second);
        } else if (mp.at(p.second).empty()) {
            mp.at(p.second).push_back("0");
            tmp.back().push_back(p.second);
        }

        if (mp.find(p.first) == mp.end()) {
            mp.insert({p.first, {mp.at(p.second).back()}});
            tmp.back().push_back(p.first);
        } else {

```

```

        mp.at(p.first).push_back(mp.at(p.second).back());
        tmp.back().push_back(p.first);
    }

    std::cout << mp.at(p.first).back() << "\n";
}
}
}

return 0;
}

```

Пояснение к примененному алгоритму:

- 1) Настраиваем ввод-вывод и создаём структуры для хранения переменных и областей видимости;
- 2) Читаем токены: { — новая область, } - выход из области (очистка переменных), x=10 или y=x — присваивание;
- 3) При присваивании: число → сохраняем в стек переменной; имя переменной → копируем последнее значение источника, выводим его;
- 4) При выходе из блока (}) — «удаляем» все переменные этой области (извлекаем верхние значения из их стеков).

Задача №D «Профессор Хаос»

```
#include <iostream>
```

```

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(nullptr);

    long long a;
    long long b;
    long long c;
    long long d;
    long long k;

```

```
std::cin >> a >> b >> c >> d >> k;
```

```
if (a * b <= c) {  
    std::cout << 0 << "\n";  
    return 0;  
}
```

```
if (a * b - c == a) {  
    std::cout << a << "\n";  
    return 0;  
}
```

```
if (a * b - c >= d) {  
    std::cout << d << "\n";  
    return 0;  
}
```

```
long long result = a * b - c;
```

```
for (long long i = 1; i < k && 0 < result && result < d; i++) {  
    result = result * b - c;  
}
```

```
if (result < 0) {  
    std::cout << 0 << "\n";  
    return 0;  
}
```

```
if (result > d) {  
    std::cout << d << "\n";  
    return 0;  
}
```



```
std::cout << result << "\n";  
return 0;  
}
```

Пояснение к примененному алгоритму:

- 1) Считать a, b, c, d, k ;
- 2) Проверить три особых случая после первого шага — возможно, сразу вывести ответ;
- 3) Итеративно обновлять $result = result * b - c$, пока не пройдёт $k-1$ шагов или $result$ не выйдет за границы $(0, d)$;
- 4) Вывести $0, d$ или итоговое значение $result$ — в зависимости от финального диапазона.